

The Rule-Authoring Test as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the **rule-authoring test** — the operational procedure that verifies a CKS deployment satisfies the rule-authoring component of the source paper's "human-governed" commitment — as a standalone procedure specification. This note completes the four-governance-rights test cluster begun by sibling notes covering the inspect-right, modify-right, and override-right tests.

Abstract

The CKS pattern's "human-governed" commitment names four governance affordances together: humans inspect, modify, and override substrate content and orchestration rules at any time, and humans author the orchestration rules that govern cell-level behavior. Sibling derivation notes formalize the inspect-, modify-, and override-right tests as standalone procedures. This note formalizes the fourth: the rule-authoring test, which verifies that authorized humans can author and revise the rules that constitute the architectural frame within which all other governance rights operate. The note specifies the architectural commitment under test, the test procedure as operational steps, the pass/fail criteria, the anti-patterns the procedure specifically detects (LLM-as-source-of-truth for rules, vendor-revocable governance over rule storage, vendor-managed rule immutability, LLM-suggested rules without human review, compliance-framework-driven rule generation, workflow-locked rule changes, and LLM-gatekeeping over rule authoring), how the procedure integrates with deployment verification, and the procedure's architectural limits. With this note the four-governance-rights test cluster is closed.

1. Why the rule-authoring test needs to be formalized as standalone

Rule authoring is the most fundamental of the four governance affordances the source paper's human-governed commitment names (§3.1, §3.3). The reason is structural. The other three rights — inspect, modify, override — operate on substrate content and on orchestration rules. The rule-authoring right is what brings those rules into existence and revises them when the operating context changes. Rules specify what cells do when they execute, how authority flows across substrate boundaries, what conflict states mean for downstream resolution, and which categories of substrate content the deployment treats as authoritative. Without rule-authoring access, humans can inspect rules they did not write, modify content the rules govern, and override outcomes the rules produced — but they cannot directly construct the architectural frame the other rights operate within. The frame is then imported from elsewhere: from a vendor's managed configuration, from a compliance framework's templated rule set, from an LLM's autonomous proposal, or from a workflow tool's lock-in. Each alternative moves rule authority away from the deployment's humans, and each compromises the human-governed commitment in a way the other three tests do not detect.

The remedy is to test the rule-authoring right specifically, as a procedure separable from the other three. This note specifies that procedure. It formalizes the most fundamental governance test as public prior art and closes the four-governance-rights test cluster — inspect, modify, override, rule-authoring — such that the cluster comprehensively tests the human-governed commitment as articulated in §3.1 and §3.3 of the source paper.

2. The architectural commitment under test

Stated operationally, the commitment is that authorized humans can author new orchestration rules and revise existing ones, where:

1. **Rules specify cell behavior** — what cells do when they execute, how cells handle inputs, what cell outputs are admissible, and how cell-level conflict resolution proceeds within the rule's scope.
2. **Rules specify authority distribution** — who can write to which substrate scope, who can override which decision class, and what authority an LLM mediator carries within cell execution.
3. **Rules specify conflict semantics** — what counts as a conflict at the cell scope, what the substrate must preserve as first-class addressable content when one arises, and how downstream resolution may proceed.
4. **Rules specify which substrate content is authoritative** — which categories carry the source-of-truth status the source paper commits to in §11.3, and which are derived or non-authoritative.

Authoring is performed without an LLM autonomously generating the rule and committing it without human authorship. Rules are stored as substrate-resident authoritative content, not as vendor configuration, compliance-framework metadata, or external-workflow-engine state where the rule's lifecycle is governed by a vendor's controls rather than by the deployment's humans. Rules become operational without requiring vendor approval, framework certification, or workflow gatekeeping beyond the authoring action itself. Rule changes produce the provenance metadata path retraceability requires (the human author of record, the time of authoring, the prior rule state on revision, and a

rationale field where deployment policy specifies one). The right is *architectural* — the deployment cannot lose it through vendor policy change — and *temporal* — exercisable at any time, not only in scheduled rule-update windows or compliance-review intervals.

3. The test procedure as operational steps

Step 1 — Author a new rule directly. An authorized human authors a new orchestration rule that governs cell behavior. The authoring is performed by the human directly, without an LLM autonomously generating the rule and committing it as substrate state without explicit human authorship. LLM drafting assistance is admissible — the human may use an LLM to draft candidate rule text, propose phrasings, or check for inconsistencies — provided the human is the architectural author of record and the rule does not become operational until the human commits it.

Step 2 — Verify substrate residency. The committed rule is stored as substrate-resident authoritative content. The evaluator inspects the substrate directly (using the inspect right verified by the sibling test) and confirms the rule is present in the substrate's structured representation, addressable by the same mechanisms that address other authoritative content. Rules stored in vendor configuration, compliance-framework metadata, workflow-engine databases, or LLM-side memory fail this step regardless of how reliably they appear available in operation.

Step 3 — Verify operational activation without external gating. The committed rule is operational from the moment of commitment — cells executing under the deployment's orchestration are governed by it. The evaluator confirms activation does not require vendor approval, framework certification, workflow sign-off, or any other gate beyond the human's authoring action.

Step 4 — Verify provenance. The rule change produces the provenance metadata path retraceability requires: the human author of record, the time of the authoring action, the prior rule state on revision, and a rationale field where policy specifies one. The evaluator confirms the provenance is present and addressable as substrate content.

Step 5 — Revise an existing rule. An authorized human revises an existing rule. The revision succeeds under the same architectural properties as new-rule authoring (Steps 1–4): direct authoring, substrate residency, operational activation without external gating, and provenance recording the revision and its predecessor state.

Step 6 — Verify temporal availability. The evaluator runs Steps 1 and 5 at a moment of the evaluator's choosing — not at a scheduled rule-update window, within a compliance-review interval, or under a workflow maintenance lockout. The action succeeds at the chosen time.

A deployment that completes all six steps without failure satisfies the rule-authoring test.

4. What the test outputs

The output is binary at the procedure level (pass/fail) and structured at the diagnostic level (which step failed, and which sub-property the failure indicates).

Pass. All six steps complete without failure. The deployment satisfies the rule-authoring component of the human-governed commitment.

Fail. One or more steps fail. The failing step identifies the violated sub-property: Step 1 indicates LLM-autonomous authoring, LLM-gatekeeping, or human authoring blocked by tooling; Step 2 indicates rules stored outside substrate; Step 3 indicates external gating prevents activation; Step 4 indicates rule changes produce inadequate provenance; Step 5 indicates rule revision is gated differently from rule origination; Step 6 indicates the right is available only in scheduled windows.

A pass indicates the rule-authoring right is architecturally available; it does not indicate the rules are correct, complete, or appropriate to the deployment's domain (§7).

5. Anti-patterns the test specifically detects

LLM-as-source-of-truth for rules. When the LLM autonomously authors or modifies rules — even with human review afterwards — authority over the rules sits on the LLM side. Human review of LLM-authored rules is a procedural overlay; the architectural author is the LLM, and the human's role is approval rather than authorship. Authority and approval are different commitments: authority constructs the frame, approval consents to a frame already constructed elsewhere. Detected at Step 1.

Vendor-revocable governance over rule storage. When rules are stored in vendor systems and the vendor retains the architectural ability to modify, revoke, or template-replace rules independent of the deployment's authoring, the right is compromised at the storage layer. The deployment's humans may "author" rules in form, but the substrate of authority is the vendor's. Detected at Step 2.

Vendor-managed rules storage. When rules live in vendor configuration rather than substrate-resident content, the deployment's humans interact with the rules through the vendor's tooling, on the vendor's lifecycle, under the vendor's terms. Detected at Step 2 and frequently at Step 3.

Vendor-managed rule immutability. When existing rules cannot be directly modified by the deployment — revision requires vendor change requests or vendor-controlled change pipelines — the right is compromised at the modification path even when origination of new rules appears to succeed. Detected at Step 5.

LLM-suggested rules without human review. When the deployment activates LLM-suggested rules through automated acceptance or default-on activation, without explicit human authorship, the LLM is the author of record. Detected at Step 1.

Compliance-framework-driven rules. When rules are determined by compliance-framework configuration — the framework specifies the rule set and the deployment's role is to install rather than to author — the architectural author is the framework's vendor. A framework that adds requirements alongside which the deployment authors its own rules is admissible; a framework that replaces the deployment's rule authority is not. Detected at Step 1 and Step 2.

Workflow-locked rule changes. When rule changes require workflow-engine approval gates beyond the authoring action — committee sign-off, multi-stage review, scheduled change windows — operational activation is gated by an external lifecycle the rule-authoring action does not control. Detected at Step 3 and Step 6.

LLM-gatekeeping for rule authoring. When the only path to rule authoring runs through an LLM call — the human asks the system to add a rule, and the LLM produces and commits it — the LLM is the

gate. Distinct from LLM-as-source-of-truth in that the human may nominally remain the author of record, but the authoring path itself is mediated by the LLM rather than direct. Detected at Step 1.

6. How the test integrates with deployment verification

Initial deployment validation. Before activation, the test runs against the deployed configuration to confirm rule authoring works end-to-end. A deployment shipping with intact rule-authoring capability has architectural cover for the commitment from the start; one shipping without it requires correction before activation.

Composition-partner verification. When a CKS deployment composes with another substrate, the test runs against the composed system to confirm rule authoring works uniformly across the composition boundary. A composition that preserves the right on each side independently but breaks it across the boundary fails this verification.

Periodic audit. Per organizational audit policy, the test runs at intervals to confirm the right has not silently regressed due to vendor policy updates, infrastructure changes, or workflow integration.

Vendor-migration verification. When the deployment migrates between vendors — substrate, LLM, or infrastructure — the test runs after migration to confirm rule-authoring is preserved. The tool-agnosticism commitment requires the right hold in any environment satisfying the three minimal requirements (§7.1); migration verification is the operational form of that commitment for this affordance.

Compliance-framework-update verification. When a compliance framework the deployment integrates with changes, the test runs to confirm the framework's update has not silently overridden rule-authoring. A framework that adds requirements is admissible; a framework that replaces the deployment's rule authority with templated content is not.

7. Limits of the test

The test is narrow by design.

It does not verify the other three rights. The inspect-, modify-, and override-right tests verify those rights specifically. Passing rule-authoring does not entail passing the other three; the four together compose the human-governed commitment, and each is verified on its own procedure.

It does not verify rule correctness. Whether the authored rules are correct, complete, or appropriate to the deployment's domain is addressable through cell-execution review, conflict-resolution audit, and other mechanisms operating on rule outcomes — not on rule-authoring access.

It does not verify provenance completeness. Step 4 confirms provenance at the rule-authoring boundary; the broader path-retraceability properties are covered by a separate operational test in this series.

It does not verify the source-of-truth categories. Step 2 confirms rules are substrate-resident; the broader source-of-truth commitment from §11.3 is covered by a separate operational test.

It does not verify whether rules produce desired behavior. Whether authored rules produce desired cell behavior is a function of the rules and the cells under operation, not a property of authoring access.

8. One-sentence test

A CKS deployment satisfies the rule-authoring component of the human-governed commitment if and only if an authorized human can, at a moment of the human's choosing, author a new orchestration rule and revise an existing one — directly, without LLM autonomous authorship, without vendor or compliance-framework gatekeeping, and without workflow approval beyond the authoring action itself — with the rule, once committed, being substrate-resident, operational, and accompanied by the provenance metadata path retraceability requires.

9. Why naming the test as standalone matters

Three consequences follow from treating the rule-authoring test as a standalone procedure specification.

It makes the most fundamental governance affordance independently testable. The other three rights operate within the frame the rules constitute. A deployment that satisfies the inspect-, modify-, and override-right tests but fails the rule-authoring test has functioning governance over a frame the deployment's humans cannot directly construct. The standalone test makes that failure mode detectable on its own procedure rather than diagnosable only through the broader behavior the imported frame produces.

It closes the four-governance-rights test cluster. Together with the sibling tests for inspect, modify, and override, the rule-authoring test provides the four operational procedures that comprehensively verify the human-governed commitment. A deployment passing all four satisfies the commitment in its full architectural content; a deployment failing any of the four fails the commitment specifically at that affordance, and remediation can be targeted precisely. Before this note, the cluster verified three rights and a fourth implied by the joint commitment but lacking its own procedure; with this note, the joint commitment has four matched procedures, and the cluster's coverage is complete.

It clears the way for the next operational-test notes. With the four-rights cluster closed, subsequent operational-test notes in this series move to other foundational commitments — the AI-as-substrate-mediator role test, the determinism-contract tests, the path-retraceability test, the source-of-truth-categories test, the tool-agnosticism test, the linear-cost test, the conflict-preservation test, the composition-requirement test, and others enumerated in this series' operational-tests phase. Each verifies one architectural commitment through a procedure analogous in form to this note's. The four-rights cluster's closure is what allows the series to advance with the human-governed commitment fully tested as architectural prior art.

Subsequent work that implements, extends, or argues against the CKS rule-authoring commitment should use "the rule-authoring test" in the sense formalized here. Subsequent work that uses the term differently is using a different procedure, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Rule-Authoring Test as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.